Fraud Detection in Highly Imbalanced Data: A Multi-Model Approach

CSCI 5502 Data Mining | Group 16 | University of Colorado Boulder | Spring 2026

BAF Dataset (Feedzai)
999,999 rows | 1.10% fraud

Project Overview

What We Built

A comprehensive fraud detection study on the Bank Account Fraud (BAF) dataset by Feedzai. With only 1.10% of 999,999 transactions being fraudulent, predicting "Legit" for everything scores 98.9% accuracy - so we built models that actually catch fraud, prioritizing Recall above all else.

10	81.78%	0.8955	4
Models tested	Best recall	Best ROC-AUC	ML categories

Project Links

GitHub: github.com/sawyermckenney/CSCI-5502-Group-16

Website: csci-5502-group-16.netlify.app

Team Contributions

Samuel Meaux	Decision Tree, SVM, Class Imbalance Handling (RUS)
Kayo Abdi	Decision Tree, K-Means Clustering, Apriori
David Savić	Decision Tree, Naive Bayes, Apriori, Logistic Regression, Ensemble
Sawyer McKenney	SVM, KNN, Normalization & Scaling, Stacking

Meet the Team

Samuel Meaux

Decision Tree · Support Vector Machine · Class Imbalance

Kayo Abdi

Decision Tree · K-Means · Apriori

David Savić

Decision Tree · Naïve Bayes · Apriori · LR · Ensemble

Sawyer McKenney

Support Vector Machine · K-Nearest Neighbour · Normalization · Stacking

Fraud Detection in Highly Imbalanced Data: A Multi-Model Approach

CSCI 5502 Data Mining | Group 16 | University of Colorado Boulder | Spring 2026

BAF Dataset (Feedzai)
999,999 rows | 1.10% fraud

Research Overview

Objective

Build fraud detection models that prioritize catching fraud (recall) over accuracy. On this dataset, predicting 'legit' for everything scores 98.9% accuracy — making recall the only metric that matters.

Class imbalance

98.9% Legitimate | 1

11,029 fraud cases out of 999,999 transactions

Why recall over accuracy?

A missed fraud = direct financial loss. A false alarm = inconvenience. The cost is asymmetric, so models are tuned to minimize false negatives above all else.

Preprocessing pipeline

- **-1 sentinel → NaN**
  prev_address & bank_months_count
- **Median imputation**
  numeric missing values
- **OneHotEncoding**
  5 categorical columns → 51 features
- **StandardScaler**
  required for SVM, KNN, LR
- **RandomUnderSampler**
  11,029 per class (oversampling timed out)
- **LassoCV selection**
  49 → 47 features (Sam's pipeline)

Models Implemented

Classification

- Decision Tree (3 implementations)
- Naive Bayes
- SVM - Linear & RBF (2 implementations)
- K-Nearest Neighbors

Regression

- Logistic Regression
- Also serves as stacking meta-learner

Clustering

- K-Means (k=3)
- Silhouette=0.076, DB index computed

Freq. Patterns

- Apriori (2 implementations)
- 687 itemsets · 105 rules · min lift=1.2

Ensemble

- Soft Voting (8 models)
- Stacking w/ LR meta-learner

Performance Results

All models — recall / F1 / ROC-AUC

Model	Recall	F1	ROC-AUC
Decision Tree	0.729	0.060	0.804
Naive Bayes	0.813	0.056	0.824
Logistic Reg.	0.787	0.081	0.877
Linear SVC	0.787	0.082	0.877
KNN	0.790	0.054	0.806
RBF SVM	0.804	0.077	0.873
Rand. Forest	0.784	0.084	0.881
Grad. Boost	0.811	0.090	0.895
Soft Voting	**0.814**	**0.080**	**0.885**
★ Stacking	**0.818**	**0.090**	**0.896**

Top 5 by recall (ensemble in green)

Key Findings

Best result: Stacking Ensemble

81.78% — **Recall**
caught 1,804 of 2,206 fraud cases

0.0897 — **F1 Score**
best balance of precision & recall

0.8955 — **ROC-AUC**
strongest class discrimination

Apriori - fraud behavior patterns

velocity6h_high + velocity24h_high
Lift: 1.94 — *Rapid multi-window transactions*

No home phone + low velocity
Lift: 1.71 — *New account, small test transactions*

No other cards + low velocity
Lift: 1.62 — *Isolated single-account profile*

Conclusions

- Stacking 8 diverse models outperformed every standalone classifier on all metrics.
- Gradient Boosting was the best individual model (F1=0.090, ROC-AUC=0.895).
- Logistic Regression held up best on the full imbalanced test set (F1=8%, Recall=80%).
- K-Means revealed fraud concentrates in one cluster (~2.24% vs ~0.8% in others).
- No feature combination was exclusively fraudulent - behavioral overlap makes this hard.

Fraud Detection in Highly Imbalanced Data: Walkthrough and Performance

CSCI 5502 Data Mining | Group 16 | University of Colorado Boulder | Spring 2026

BAF Dataset (Feedzai)
999,999 rows | 1.10% fraud

Preprocessing Overview

Data Loading

```python
# ========================
# LOAD DATA
# ========================

key_path = "../../../fraud-detection-key.json"

if os.path.exists(key_path):
    client = bigquery.Client.from_service_account_json(key_path)
    query = "SELECT * FROM `csci-4022.CSCI.fraud_data`"
    print("Fetching dataset...")
    df = client.query(query).to_dataframe()
    print(f"Loaded {len(df):,} rows.")
else:
    raise FileNotFoundError("BigQuery key not found at: " + key_path)
```

BigQuery Utilizaiton

Due to the size limitations of GitHub, it was imperative to utilize BigQuery to access the raw dataset which consisted of 999,999 transactions.

Data Cleaning

```python
# columns where -1 means missing
maybe_missing_cols = [
    "prev_address_months_count",
    "bank_months_count"
]

# replace -1 with NaN
for col in maybe_missing_cols:
    df[col] = df[col].replace(-1, np.nan)

print("Shape:", df.shape)
print("\nFraud distribution:\n",
      df["fraud_bool"].value_counts(normalize=True))
print("\nTop missing values:\n",
      df.isna().sum().sort_values(ascending=False).head(10))
```

Missing Values

Large amounts of missing data were prevalent in two features. It was determined that missing values might play a statistically significant role so those values were preserved with "NaN" values

Models and Metrics

Model Selection

The root challenge of the problem was detecting fraud in a highly unbalanced dataset. As such the following models were selected:

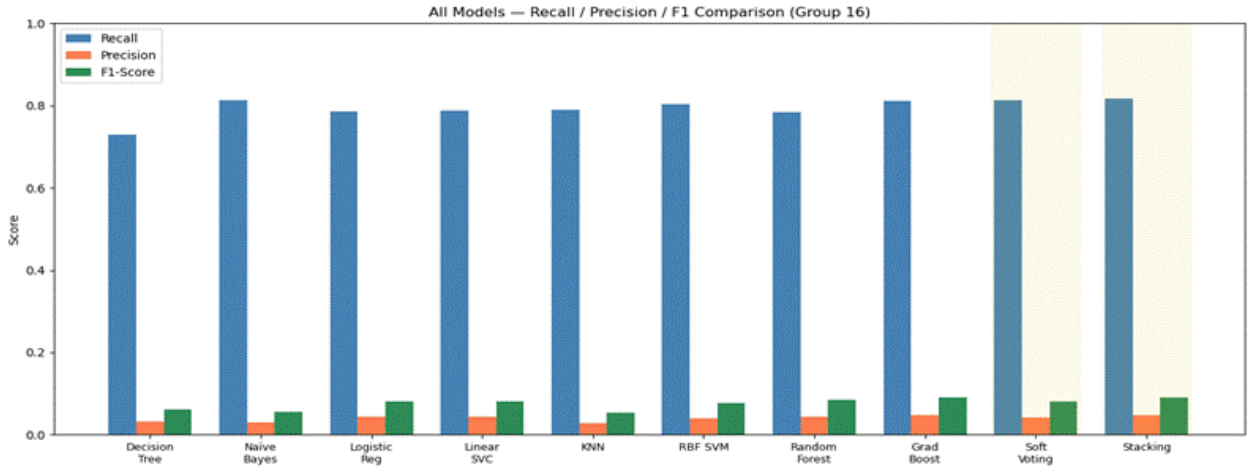
- Decision Trees
- Support Vector Machines (SVM)
- K-Nearest Neighbors (KNN)
- Stacking and Ensemble Methods
- K-Means Clustering
- Apriori Rule Mining

Performance Evaluation Metrics

In terms of fraud detection, a model with 98.9% accuracy is misleading. Therefore, the following metrics were evaluated:

- Recall
- Precision
- F1-Score
- Accuracy
- Confusion Matrices

Performance Results



- Stacking yielded the highest F1 of 0.0897 along with highest ROC-AUC of 0.8955, and highest recall of 81.78%.
- Gradient Boosting and Logistic Regression outputs were weighed more heavily from the meta-learner, as the two had strongest individual performance.
- Soft voting with recall of 81.37% and ROC-AUC of 0.8846.
- Both ensemble methods outperformed every individual model on every metric, and that is confirmation that merging the approaches is far better than relying on any of the ones as single method.

Key Findings

1. Meta-Learning: Combining all base models outperformed any single algorithm. A peak Recall of 81.78% was realized.
2. Logistic Regression was the most reliable model for "real-world" imbalanced sets. A peak F1-Score of 8% while maintaining 80% recall was achieved.
3. Fraud was not driven by single factors. Housing status and device types along with velocity and session activity were identified as top predictors.
4. Model 3 (Stacking Ensemble) was realized as the ultimate model recommendation as it maximizes fraud detection (81.78% Recall) while using a meta-learning framework to identify the most probable outcomes.